

Training Classification

Contents

- Training Classification
- Import packages
- Change data inputs here
- Change Message-Passing Neural Network (MPNN) inputs here
- Set up trainer
- Start training
- Test results

 [Open in Colab](#)

```
[1]: # Install chemprop from GitHub if running in Google Colab
import os

if os.getenv("COLAB_RELEASE_TAG"):
    try:
        import chemprop
    except ImportError:
        !git clone https://github.com/chemprop/chemprop.git
        %cd chemprop
        !pip install .
        %cd examples
```

```
[2]: import pandas as pd
from pathlib import Path

from lightning import pytorch as pl

from chemprop import data, featurizers, models, nn
```

```
[3]: chemprop_dir = Path.cwd().parent
input_path = chemprop_dir / "tests" / "data" / "classification" / "mol.cs
num_workers = 0 # number of workers for dataloader. 0 means using main pr
smiles_column = 'smiles' # name of the column containing SMILES strings
target_columns = ['NR-Ahr', 'NR-ER', 'SR-ARE', 'SR-MMP'] # classification
```

Load data

```
[4]: df_input = pd.read_csv(input_path)
df_input
```

[4]:

	smiles	NR-AhR	NR-ER	SR-ARE	SR-MMP
0	<chem>CCOc1ccc2nc(S(N)(=O)=O)sc2c1</chem>	1.0	NaN	1.0	0.0
1	<chem>CCN1C(=O)NC(c2ccccc2)C1=O</chem>	0.0	0.0	NaN	0.0
2	<chem>CC[C@]1(O)CC[C@H]2[C@@H]3CCC4=CCCC[C@@H]4[C@H]...</chem>	NaN	NaN	0.0	NaN
3	<chem>CCCN(CC)C(CC)C(=O)Nc1c(C)cccc1C</chem>	0.0	0.0	NaN	0.0
4	<chem>CC(O)(P(=O)(O)O)P(=O)(O)O</chem>	0.0	0.0	0.0	0.0
...	...	...	...	...	...
495	<chem>Cc1cccc1CO[C@H]1C[C@]2(C(C)C)CC[C@@]1(C)O2</chem>	NaN	0.0	0.0	0.0
496	<chem>NNc1ccc(C(=O)O)cc1</chem>	NaN	NaN	0.0	0.0
497	<chem>CCCCCOc1cccc1C(=O)O</chem>	0.0	NaN	0.0	0.0
498	<chem>O=C(OCc1cccc1)C(=O)OCc1cccc1</chem>	0.0	0.0	0.0	0.0
499	<chem>CCCS1ccc2[nH]c(NC(=O)OC)nc2c1</chem>	1.0	1.0	0.0	1.0

500 rows × 5 columns

Get SMILES and targets

```
[5]: smis = df_input.loc[:, smiles_column].values
ys = df_input.loc[:, target_columns].values
```

```
[6]: # Take a look at the first 5 SMILES strings and target columns
smis[:5], ys[:5]
```

```
[6]: (array(['CCOc1ccc2nc(S(N)(=O)=O)sc2c1', 'CCN1C(=O)NC(c2ccccc2)C1=O',
            'CC[C@]1(O)CC[C@H]2[C@@H]3CCC4=CCCC[C@@H]4[C@H]3CC[C@@]21C',
            'CCCN(CC)C(CC)C(=O)Nc1c(C)cccc1C', 'CC(O)(P(=O)(O)O)P(=O)(O)O'],
      dtype=object),
      array([[ 1., nan,  1.,  0.],
            [ 0.,  0., nan,  0.],
            [nan, nan,  0., nan],
            [ 0.,  0., nan,  0.],
            [ 0.,  0.,  0.,  0.])))
```

Get molecule datapoints

```
[7]: all_data = [data.MoleculeDatapoint.from_smi(smi, y) for ...]
[22:09:11] WARNING: not removing hydrogen atom without neighbors
```

Perform data splitting for training, validation, and testing

```
[8]: # available split types
list(data.SplitType.keys())
```

```
[8]: ['SCAFFOLD_BALANCED',
      'RANDOM_WITH_REPEATED_SMILES',
      'RANDOM',
      'KENNARD_STONE',
      'KMEANS']
```

```
[9]: mols = [d.mol for d in all_data] # RDkit Mol objects are use for structu
train_indices, val_indices, test_indices = data.make_split_indices(mols,
train_data, val_data, test_data = data.split_data_by_indices(
    all_data, train_indices, val_indices, test_indices
)
```

The return type of make_split_indices has changed in v2.1 - see help(make_

Get MoleculeDataset

```
[10]: featurizer = featurizers.SimpleMoleculeMolGraphFeaturizer()

train_dset = data.MoleculeDataset(train_data[0], featurizer)
val_dset = data.MoleculeDataset(val_data[0], featurizer)
test_dset = data.MoleculeDataset(test_data[0], featurizer)
```

Get DataLoader

```
[11]: train_loader = data.build_dataloader(train_dset, num_workers=num_workers)
val_loader = data.build_dataloader(val_dset, num_workers=num_workers, shu
test_loader = data.build_dataloader(test_dset, num_workers=num_workers, s
```

Message Passing

A `Message passing` constructs molecular graphs using message passing to learn node-level hidden representations.

Options are `mp = nn.BondMessagePassing()` or `mp = nn.AtomMessagePassing()`

 [main](#) ▼

```
[12]: mp = nn.BondMessagePassing()
```

Aggregation

An `Aggregation` is responsible for constructing a graph-level representation from the set of node-level representations after message passing.

Available options can be found in `nn.agg.AggregationRegistry`, including

- `agg = nn.MeanAggregation()`
- `agg = nn.SumAggregation()`
- `agg = nn.NormAggregation()`

```
[13]: print(nn.agg.AggregationRegistry)
ClassRegistry {
  'mean': <class 'chemprop.nn.agg.MeanAggregation'>,
  'sum': <class 'chemprop.nn.agg.SumAggregation'>,
  'norm': <class 'chemprop.nn.agg.NormAggregation'>
}
```

```
[14]: agg = nn.MeanAggregation()
```

Feed-Forward Network (FFN)

A `FFN` takes the aggregated representations and make target predictions.

Available options can be found in `nn.PredictorRegistry`.

For regression:

- `ffn = nn.RegressionFFN()`
- `ffn = nn.MveFFN()`
- `ffn = nn.EvidentialFFN()`

For classification:

- `ffn = nn.BinaryClassificationFFN()`
- `ffn = nn.BinaryDirichletFFN()`
- `ffn = nn.MulticlassClassificationFFN()`
- `ffn = nn.MulticlassDirichletFFN()`

For spectral:

- `ffn = nn.SpectralFFN()` # will be available in future version

```
[15]: print(nn.PredictorRegistry)

ClassRegistry {
  'regression': <class 'chemprop.nn.predictors.RegressionFFN'>,
  'regression-mve': <class 'chemprop.nn.predictors.MveFFN'>,
  'regression-evidential': <class 'chemprop.nn.predictors.EvidentialFFN'>,
  'regression-quantile': <class 'chemprop.nn.predictors.QuantileFFN'>,
  'classification': <class 'chemprop.nn.predictors.BinaryClassificationFFN'>,
  'classification-dirichlet': <class 'chemprop.nn.predictors.BinaryDirichletFFN'>,
  'multiclass': <class 'chemprop.nn.predictors.MulticlassClassificationFFN'>,
  'multiclass-dirichlet': <class 'chemprop.nn.predictors.MulticlassDirichletFFN'>,
  'spectral': <class 'chemprop.nn.predictors.SpectralFFN'>
}

[16]: ffn = nn.BinaryClassificationFFN(n_tasks = len(target_columns))
```

Batch Norm

A `Batch Norm` normalizes the outputs of the aggregation by re-centering and re-scaling.

Whether to use batch norm

```
[17]: batch_norm = False
```

Metrics

`Metrics` are the ways to evaluate the performance of model predictions.

Available options can be found in `metrics.MetricRegistry`, including

```
[18]: print(nn.metrics.MetricRegistry)
```

```

ClassRegistry {
    'mse': <class 'chemprop.nn.metrics.MSE'>,
    'mae': <class 'chemprop.nn.metrics.MAE'>,
    'rmse': <class 'chemprop.nn.metrics.RMSE'>,
    'bounded-mse': <class 'chemprop.nn.metrics.BoundedMSE'>,
    'bounded-mae': <class 'chemprop.nn.metrics.BoundedMAE'>,
    'bounded-rmse': <class 'chemprop.nn.metrics.BoundedRMSE'>,
    'r2': <class 'chemprop.nn.metrics.R2Score'>,
    'binary-mcc': <class 'chemprop.nn.metrics.BinaryMCCMetric'>,
    'multiclass-mcc': <class 'chemprop.nn.metrics.MulticlassMCCMetric'>,
    'roc': <class 'chemprop.nn.metrics.BinaryAUROC'>,
    'prc': <class 'chemprop.nn.metrics.BinaryAUPRC'>,
    'accuracy': <class 'chemprop.nn.metrics.BinaryAccuracy'>,
    'f1': <class 'chemprop.nn.metrics.BinaryF1Score'>
}

```

```

[19]: # AUROC used by default
metric_list = None

```

Constructs MPNN

```

[20]: mpnn = models.MPNN(mp, agg, ffn, batch_norm, metric_list)

mpnn

```

```

[20]: MPNN(
  (message_passing): BondMessagePassing(
    (W_i): Linear(in_features=86, out_features=300, bias=False)
    (W_h): Linear(in_features=300, out_features=300, bias=False)
    (W_o): Linear(in_features=372, out_features=300, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
    (tau): ReLU()
    (V_d_transform): Identity()
    (graph_transform): Identity()
  )
  (agg): MeanAggregation()
  (bn): Identity()
  (predictor): BinaryClassificationFFN(
    (ffn): MLP(
      (0): Sequential(
        (0): Linear(in_features=300, out_features=300, bias=True)
      )
      (1): Sequential(
        (0): ReLU()
        (1): Dropout(p=0.0, inplace=False)
        (2): Linear(in_features=300, out_features=4, bias=True)
      )
    )
  )
  (criterion): BCELoss(task_weights=[[1.0, 1.0, 1.0, 1.0]])
  (output_transform): Identity()
)
(X_d_transform): Identity()
(metrics): ModuleList(
  (0): BinaryAUROC()
  (1): BCELoss(task_weights=[[1.0, 1.0, 1.0, 1.0]])
)
)

```

```
[21]: trainer = pl.Trainer(
    logger=False,
    enable_checkpointing=True, # Use `True` if you want to save model che
    enable_progress_bar=True,
    accelerator="cpu",
    devices=1,
    max_epochs=20, # number of epochs to train for
)
```

💡 Tip: For seamless cloud uploads and versioning, try installing [litmodelhub]
 GPU available: False, used: False
 TPU available: False, using: 0 TPU cores

```
[22]: trainer.fit(mpnn, train_loader, val_loader)
```

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning:
 Loading `train\_dataloader` to estimate number of stepping batches.
 /home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning:

	Name	Type	Params	Mode	FLOPs
0	message_passing	BondMessagePassing	227 K	train	0
1	agg	MeanAggregation	0	train	0
2	bn	Identity	0	train	0
3	predictor	BinaryClassificationFFN	91.5 K	train	0
4	X_d_transform	Identity	0	train	0
5	metrics	ModuleList	0	train	0

Trainable params: 319 K

Non-trainable params: 0

Total params: 319 K

Total estimated model params size (MB): 1

Modules in train mode: 24

Modules in eval mode: 0

Total FLOPs: 0

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning:
 tor.py:434: The 'val_dataloader' does not have many workers which may be
 value of the `num\_workers` argument` to `num_workers=11` in the `DataLoa

`Trainer.fit` stopped: `max\_epochs=20` reached.

```
[23]: results = trainer.test(mpnn, test_loader, weights_only=False) # weights_
```

Test metric	DataLoader 0
test/roc	0.6451334357261658

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning: